Attention'dan Multi-Head Attention'a: Neden Tek Bir Bakış Açısı Yetmedi?

Atıl Samancıoğlu

1 Bu Dökümanda Ne Yapıyoruz?

Bir önceki dökümanda şuraya kadar geldik:

> Attention, bir kelimenin diğer kelimelere doğrudan bakabilmesini sağlar.

Artık elimizde net bir mekanizma var: Query, Key ve Value vektörleriyle kelimeler arası ilişkileri ölçebiliyoruz. Hatta bunu paralel olarak da yapabiliyoruz.

Bu noktada çok doğal bir soru ortaya çıkar:

> "Bu kadar güçlü bir mekanizma varken, neden yetmedi?"

Bu dökümanda tam olarak bu sorunun cevabını vereceğiz. Tek bir attention hesabının **hangi açıdan sınırlı kaldığını** ve bu sınırların **Multi-Head Attention** ile nasıl aşıldığını adım adım inşa edeceğiz.

2 Ana Örneğimiz Hâlâ Bizimle

Yine aynı cümleyle devam ediyoruz:

> *"I did not say he stole the money."*

Bu sefer cümleye biraz daha dikkatli bakalım.

Bu cümlede aynı anda birden fazla şey oluyor:

- "stole" ile "money" arasında **eylem–nesne** ilişkisi
- "not" ile fiil arasında **olumsuzluk** ilişkisi
- "I" ile "say" arasında **özne–eylem** ilişkisi

Yani tek bir kelime, başka kelimelerle **farklı açılardan** ilişki kuruyor.

İşte kritik soru burada doğuyor.

3 Tek Bir Attention Ne Yapar?

Tek-head attention kullandığımızda şunu yapıyoruz:

- Her kelime için tek bir Query
- Tek bir Key uzayı
- Tek bir Value uzayı

Yani model, kelimeler arası ilişkileri **tek bir temsil uzayında** öğrenmek zorunda kalıyor.
Bu şu anlama gelir:

> "Model, aynı anda hem anlam ilişkisini, hem gramer ilişkisini, hem de bağlamsal vurguyu tek bir bakış açısına sıkıştırıyor."

Bu, özellikle karmaşık cümlelerde ciddi bir kısıt.

3.1 Sezgisel Bir Benzetme

Şöyle düşün:
Bir cümleye bakarken bazen şu soruyu sorarsın:

> "Bu cümlenin öznesi kim?"

Bazen de şunu:

> "Bu cümlede asıl vurgu nerede?"

Aynı cümle, ama **farklı zihinsel lensler**.
Tek-head attention, bu lenslerin hepsini tek bir gözlükle takmaya çalışır.
Multi-Head Attention ise der ki:

> "Birden fazla gözlük takalım."

4 Multi-Head Attention Fikrinin Doğuşu

Multi-Head Attention'ın temel fikri şudur:

> Aynı cümleye, aynı anda, farklı temsil uzaylarından bakalım.

Bunu yapmak için Attention mekanizmasını kopyalarız. Ama kopyalarken küçük bir farkla:

- Her head, farklı projeksiyon matrisleri kullanır
- Yani farklı Q, K, V üretir

5 Matematiksel Yapı (Ama Kontrollü)

Girdi embedding boyutumuz d_{model} olsun. Örneğin $d_{model} = 512$.

Diyelim ki $h = 8$ adet attention head kullanıyoruz.

Her head için:

$$Q_i = XW_i^Q, \quad K_i = XW_i^K, \quad V_i = XW_i^V$$

Burada önemli olan nokta şu:

- W_i^Q, W_i^K, W_i^V her head için **farklı**
- Boyutlar genellikle $d_k = d_v = \frac{d_{model}}{h}$

Yani her head, daha küçük ama **özelleşmiş** bir uzayda çalışır.

5.1 Bu Neden Mantıklı?

Çünkü bu sayede:

- Bir head, özne–fiil ilişkisine odaklanabilir
- Bir head, olumsuzluklara duyarlı olabilir
- Bir head, uzun mesafeli bağlamı yakalayabilir

Model bunu biz söylemeden, **öğrenerek** yapar.

6 Her Head İçin Attention Hesabı

Her head için bildiğimiz attention hesabını yaparız:

$$\text{head}_i = \text{softmax}\left(\frac{Q_i K_i^\top}{\sqrt{d_k}}\right) V_i$$

Burada yeni hiçbir şey yok. Yeni olan tek şey, bunu **birden fazla kez** yapmamız.

7 Head'leri Birleştirmek

Tüm head'leri hesapladıktan sonra şu adımı yaparız:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h) W^O$$

Bu adım çok kritiktir.

7.1 Neden Concat + Linear?

Çünkü:

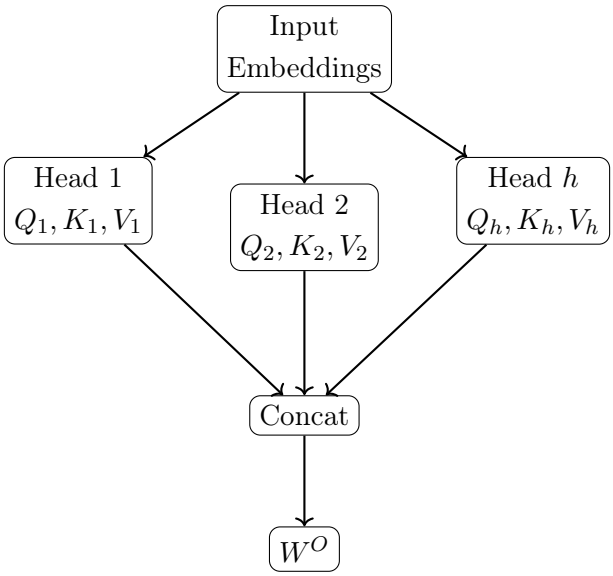
- Her head farklı bir şey öğrenmiştir
- Ama modelin geri kalanı tek bir vektör bekler

Concat işlemi bilgileri yan yana koyar, W^O ise bu bilgileri tekrar **anlamlı bir bütün** haline getirir.

Bu noktada şunu net söyleyebiliriz:

> Multi-Head Attention, "daha fazla attention" değil, "daha zengin attention" demektir.

8 TikZ ile Kavramsal Bir Görsel



Şekil 1: Multi-Head Attention'ın kavramsal akışı

Bu görseli incelerken şuna dikkat et: Input aynı, bakış açıları farklı.

9 Ana Örneğe Son Bir Kez Bakalım

"money" kelimesi için:

- Bir head "stole" ile güçlü bağ kurabilir
- Bir head "not" kelimesinin etkisini yakalayabilir
- Bir head, cümlenin genel tonunu öğrenebilir

Ve tüm bu bilgiler tek bir vektörde birleşir.

10 Burada Durup Büyük Resmi Netleştirelim

Bu dökümanın sonunda şu fikir net olmalı:

> Multi-Head Attention, aynı veriye farklı açılardan bakabilme yeteneğidir.

Bu, Transformer mimarisinin neden bu kadar güçlü olduğunun en önemli sebeplerinden biridir.

11 Transformer'da Atlanan Ama Kritik Detaylar

Bu bölümde, *Attention Is All You Need* makalesinde yer alan fakat ilk öğrenirken genellikle ya hızlı geçilen ya da hiç anlatılmayan bazı **kritik kavramları** ele alacağız.

Bu konular özellikle:

- Modelin geleceği neden "görememesi" gerektiğini,
- Attention'ın neden ölçeklendiğini,

anlamak için hayati öneme sahiptir.

12 Masking Neden Gerekti?

Önce çok temel bir soru soralım:

> **Bir dil modeli, bir kelimeyi tahmin ederken gelecekteki kelimeleri görebilir mi?**

Cevap net bir şekilde: **Hayır.**

Örneğin şu cümleyi düşünelim:

`"I am eating _____"`

Eğer model boşluğu doldururken `pizza` kelimesini zaten görüyorsa, bu bir **hile** olur. Model gerçekte dili öğrenmiş olmaz, sadece cevabı kopyalamış olur.

İşte **masking** tam olarak bu problemi çözmek için vardır.

Attention mekanizmasında her kelime, diğer tüm kelimelere bakabilir. Eğer hiçbir önlem almazsak:

- Decoder içindeki bir kelime,
- Henüz üretilmemiş kelimelere de attention uygulayabilir

Bu yüzden bazı bağlantıları **bilerek kapatmamız** gerekir. Bu kapatma işlemi matematiksel olarak **masking** ile yapılır.

13 Causal Mask vs Padding Mask

Transformer'larda iki farklı masking türü vardır ve bunlar **farklı problemleri** çözer.

13.1 Causal (Look-Ahead) Mask

Causal mask'in amacı şudur:

> **Bir kelime, sadece kendisinden önce gelen kelimelere bakabilsin.**

Örnek bir attention skor matrisi düşünelim:

$$\begin{bmatrix} * & \times & \times & \times \\ * & * & \times & \times \\ * & * & * & \times \\ * & * & * & * \end{bmatrix}$$

Burada:

- $*$: bakılmasına izin var
- $\times$: bakılması yasak

Üst üçgen kısım maskelenir. Matematiksel olarak bu hücrelere:

$$-\infty$$

eklenir ve softmax sonrası bu ağırlıklar **0** olur.

Causal mask yalnızca decoder'da kullanılır.

13.2 Padding Mask

Padding mask ise tamamen farklı bir problemi çözer.

Batch içinde cümle uzunlukları farklıdır:

```
I like pizza
I like <PAD>
```

`<PAD>` bir kelime değildir. Anlam taşımaz.

Eğer attention bu token'a bakarsa:

- anlamsız bilgi öğrenilir
- model kararsızlaşır

Bu yüzden padding token'lara ait attention skorları da maskelenir.

Özetle:

Mask Türü	Amaç	Nerede Kullanılır
Causal Mask	Geleceği gizlemek	Decoder Self-Attention
Padding Mask	Anlamsız token'ları gizlemek	Encoder & Decoder